

提高级 CSP-S 第 15 套初赛模拟试题

一、单项选择题(共 15 题,每题 2 分,共计 30 分;每题有且仅有一个正确选项)

- 下列属于图像文件格式的有()。
A. WMV B. MPEG C. JPEG D. AVI
- 以下哪一位是图灵奖得主?()
A. 特朗普 B. 拜登 C. 图灵 D. 姚期智
- 欧拉图 G 是指可以构成一个闭回路的图,且图 G 的每一条边恰好在这个闭回路上出现一次(即一笔画成)。在以下各个描述中,不一定是欧拉图的是()。
A. 图 G 中没有度为奇数的顶点
B. 包含欧拉环游的图(欧拉环游是指通过图中每边恰好一次的闭路径)
C. 包含欧拉闭迹的图(欧拉闭迹是指通过图中每边恰好一次的路径)
D. 存在一条回路,通过每个顶点恰好一次
- 设 $A=B=\text{true}$, $C=D=\text{false}$, 以下逻辑运算表达式值为 false 的是()。
A. $(\neg A \wedge B) \vee (C \wedge D \vee A)$ B. $\neg (((A \wedge B) \vee C) \wedge D)$
C. $A \wedge (B \vee C \vee D) \vee D$ D. $(A \wedge (D \vee C)) \wedge B$
- 以下图中一定可以进行黑白染色的是()。(黑白染色:为各个结点分别指定黑白两种颜色之一,使相邻结点颜色不同)
A. 基环树 B. 完全图 C. 树 D. 弦图
- 与二进制小数 0.1 相等的十六进制数是()。
A. 0.8 B. 0.4 C. 0.2 D. 0.1
- TCP 协议属于哪一层协议()。
A. 应用层 B. 传输层 C. 网络层 D. 数据链路层
- 汇编语言()。
A. 是一种与具体硬件无关的程序设计语言
B. 和汉语、英语并称为世界三大语言
C. 可以直接访问寄存器、内存单元、I/O 端口
D. 随着高级语言的诞生,如今已被完全淘汰,不再使用
- 某程序有一个大小为 N 的 int 数组 $a[N]$, 程序对这个数组做 $M(M < N)$ 次赋值操作,每次操作将均匀随机一个 $[1, N]$ 的正整数 X , 以及一个 int 范围内整数 Y , 将所有 $a[X \text{ 的倍数}]$ 赋值为 Y 。请你计算出该程序的期望时间复杂度()。
A. $O(M \log_e N)$ B. $O(M \log_2 M)$ C. $O(N \ln N)$ D. $O(MN)$
- 下列二进制数中,与二进制数 1101001 进行逻辑异或运算后最大的是()。
A. 0011010 B. 0010111 C. 1011001 D. 1000111
- 下列问题目前不属于 P 问题的是()。
A. 最长公共子串问题 B. 旅行商问题
C. 二维凸包问题 D. 最长上升子序列问题
- 假设买彩票时你需要填一个 6 位数,而中奖号码也将是一个 6 位数,如果你填的数等于中奖号码则得一等奖;如果两个数恰有一位不相同则得二等奖;如果两个数恰有两位不同则得三等奖。问你得奖的概率是多少(本题中的 6 位数允许有占位的前导 0)()。
A. $\frac{13}{40000}$ B. $\frac{111}{1000000}$ C. $\frac{127}{100000}$ D. $\frac{17}{40000}$
- 表达式 $(7 \div 5 \% 4 + 1)$ 的值是()。
A. 2 B. 3 C. 5 D. 7
- 下列程序设计语言中,不属于面向对象的语言是()。
A. C++ B. Visual Basic C. Java D. C

15. 如果在某个进制下等式 $7 * 7 = 41$ 成立, 那么在该进制下等式 $12 * 12 = (\quad)$ 也成立。

- A. 100 B. 144 C. 164 D. 196

二、阅读程序(程序输入不超过数组或字符串定义的范围,判断正确的填√,错误的填×;除特殊说明外,判断题每题 1.5 分,选择题每题 3 分,共计 40 分)

1.

```
01 #include<bits/stdc++.h>
02 using namespace std;
03 int n,ans=0;
04 int a[21],sum[21];
05 void print(){
06     ans++;
07     for(int i=0;i<n;i++)cout<<a[i]<<' ';
08     puts("");
09 }
10 void check(){
11     memset(sum,0,sizeof(sum));
12     for(int i=0;i<n;i++)sum[a[i]]++;
13     for(int i=0;i<n;i++)if(a[i]!=sum[i])return;
14     print();
15 }
16 void dfs(int x){
17     for(int i=0;i<=n;i++){
18         a[x]=i;
19         if(x==n-1)check();
20         else dfs(x+1);
21     }
22 }
23 int main(){
24     cin>>n;
25     dfs(0);
26     cout<<ans;
27     return 0;
28 }
```

●判断题

- (1) 将第 7 行的 $a[i]$ 替换为 $sum[i]$, 则输出会变化。()
 (2) 执行 $print()$ 函数时 $a[0]$ 一定为正整数。()
 (3) 执行 $print()$ 操作时 $a[0] \sim a[n-1]$ 的和等于 n 。()
 (4) 当且仅当 $n \leq 3$ 时(n 为正整数), 输出的 ans 值为 0。()

●单选题

- (5) 该程序的时间复杂度最接近于()。
 A. $O(n^2)$ B. $O(n \log n)$ C. $O(n!)$ D. $O(n)$
 (6) 输出的 ans 值最大为()。
 A. 1 B. 2 C. 4 D. 5

2.

```
01 #include<iostream>
02 using namespace std;
03 int n,a[1000];
```

```

04 int main()
05 {
06     cin>>n;
07     for (int i=0;i<n;i++) cin>>a[i];
08     int sum=0;
09     for (int i=0;i<n;i++)
10         for (int j=i+1;j<n;j++)
11             if (a[i]<a[j]) sum++;
12     cout<<sum<<endl;
13     return 0;
14 }

```

保证输入的数均为不超过 10000 的正整数,且 a 中的数互不相同。

●判断题:

- (1)当输入的 n 大于 1000 时,程序可能因为数组越界而运行错误。()
- (2)该程序求出了 a 数组的逆序对个数。()
- (3)(2 分)程序运行至 12 行时,sum 的值不可能为 0。()

●单选题:

- (4)该算法的时间复杂度为()。

A. $O(n)$	B. $O(n \log n)$	C. $O(n^2)$	D. $O(n \log^2 n)$
-----------	------------------	-------------	--------------------
- (5)将第 10 行中 $j=i+1$ 改为下列哪句话时,程序输出不可能改变()

A. $j=0$	B. $j=i$	C. $j=i-1$	D. $j=n$
----------	----------	------------	----------
- (6)(4 分)当输入为 5 \n 4 2 3 5 1 \n 时,输出为()。(其中 \n 为换行符)

A. 3	B. 4	C. 5	D. 6
------	------	------	------

3.

```

01 #include<bits/stdc++.h>
02 using namespace std;
03 const int dir[4][2]={{0,1},{1,0},{0,-1},{-1,0}};
04 bitset<1602>a[1602];bitset<1602>b[1602];
05 int n,m,len;
06 inline void qz(int x,int y){
07     for(int k=0;k<4;k++){
08         int tx=x+dir[k][0];int ty=y+dir[k][1];
09         if(tx<1||ty<1||tx>n||ty>m)continue;
10         b[(x-1)*m+y][(tx-1)*m+ty]=0;
11         b[(tx-1)*m+ty][(x-1)*m+y]=0;b[(tx-1)*m+ty][len+1]=1;
12     }
13     b[(x-1)*m+y][len+1]=1;
14 }
15 inline bool chk(){
16     for(int i=1;i<=n;i++){
17         for(int j=1;j<=m;j++){
18             int cnt=b[(i-1)*m+j][len+1];
19             for(int k=0;k<4;k++){
20                 int tx=i+dir[k][0];int ty=j+dir[k][1];
21                 if(tx<1||ty<1||tx>n||ty>m)continue;
22                 cnt+=b[(tx-1)*m+ty][len+1];
23             }

```

```
24     if(cnt&1) return false;
25     }
26 }
27 return true;
28 }
29
30 int main() {
31     srand(time(NULL));
32     scanf("%d%d", &n, &m);
33     for(int i=1; i<=n; i++) {
34         for(int j=1; j<=m; j++) {
35             a[(i-1)*m+j][(i-1)*m+j]=1;
36             for(int k=0; k<4; k++) {
37                 int tx=i+dir[k][0]; int ty=j+dir[k][1];
38                 if(tx<1 || ty<1 || tx>n || ty>m) continue;
39                 a[(i-1)*m+j][(tx-1)*m+ty]=1;
40             }
41         }
42     }
43
44     len=n*m; bool flag=false;
45     while(flag==false) {
46         int x=rand()%n+1; int y=rand()%m+1;
47         for(int i=1; i<=len; i++) {
48             for(int j=1; j<=len+1; j++) b[i][j]=a[i][j];
49         }
50
51         qz(x, y); flag=true;
52         for(int i=1; i<=len; i++) {
53             int mx_pos=0;
54             for(int j=i; j<=len; j++) {
55                 if(b[j][i]>b[mx_pos][i]) mx_pos=j;
56             }
57             swap(b[i], b[mx_pos]);
58
59             for(int j=1; j<=len; j++) {
60                 if(i==j || !b[j][i]) continue;
61                 b[j]=(b[j]^b[i]);
62             }
63         }
64         if(!chk()) flag=false;
65     }
66     for(int i=1; i<=n; i++) {
67         for(int j=1; j<=m; j++) {
68             printf("%d", (int)b[(i-1)*m+j][len+1]);
69             putchar((j==m)? '\n':");
70         }
```

```
71 }
72 return 0;
73 }
```

保证输入的 n, m 是不大于 40 的正整数。

● 判断题

- (1) 程序可能陷入死循环。()
- (2) 程序可能会输出一个全 1 矩阵。()
- (3) 将语句“`printf("%d", (int)b[(i-1)*m+j][len+1]);`”中的“(int)”去掉的话, 程序不会出现编译错误。()
- (4) 对于 $n=5, m=5$ 的输入, 输出的矩阵的第 3 行第 4 列一定为 0。()

● 单选题

- (5) 当 $n=2, m=39$ 时, 有()种本质不同的矩阵合法(即 `chk()` 函数会返回 true)且 1 的个数为 40。

A. 0 B. 1 C. 2 D. 3

- (6) (4 分) 对于 $n=3, m=5$ 的输入, 若将语句“`qz(x, y)`”改为“`qz(3, 2)`”下列哪个矩阵是可能的输出()。

A. $\begin{matrix} 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 & 1 \end{matrix}$ B. $\begin{matrix} 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 \end{matrix}$ C. $\begin{matrix} 0 & 0 & 1 & 1 & 1 \\ 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 0 \end{matrix}$ D. 没有输出, 程序会陷入死循环

三、完善程序(单选题, 每小题 3 分, 共计 30 分)

1. (放鞭炮) 有一条长度为 n 的走廊, 我们把所有位置编号为 $1, 2, 3, \dots, n$ 。有一个警察在 b 处和一个放鞭炮的人在 a 处, 放鞭炮的人手里有 m 个鞭炮, 每个鞭炮有一个爆炸时间 $s[i]$, 每一秒放鞭炮的人可以向左或右移动 1 格, 也可以选择放一个鞭炮, 鞭炮爆炸需要时间 $s[i]$, 而警察每秒只会向放鞭炮的人走 1 格, 问警察抓住他的那一秒结束前最多能有多少个鞭炮炸掉。

数据范围: $a, b \leq n \leq 1000000000$, $m \leq 200000$, $s[i] \leq 1000000000$ 。

试补全程序。

```
01 #include <bits/stdc++.h>
02 #define LL long long
03 #define pb push_back
04 #define SZ(x) ((int)x.size()-1)
05 #define ms(a,b) memset(a,b,sizeof a)
06 #define F(i,a,b) for (int i=(a); i<=(b); ++i)
07 #define DF(i,a,b) for (int i=(a); i>=(b); --i)
08 using namespace std;
09 inline int read() {
10     char ch=getchar(); int w=1, c=0;
11     or(!isdigit(ch); ch=getchar()) if (ch=='-') w=-1;
12     for(; isdigit(ch); ch=getchar()) c=(c<<1)+(c<<3)+(ch^48);
13     return w*c;
14 }
15 const int M=2e5+10;
16 int n,m,a,b,s[M];
17 void work() {
18     n=read(); m=read(); a=read(); b=read();
19     if (a>b) {
20         a = ①;
```

```

21     b=n-b+1;
22 }
23 F(i,l,m){
24     s[i]=read();
25 }
26 int tim= ②, num=min(b-a-1,m);
27 sort(s+1,s+m+1);
28 int l=0,r=num,ans=0;
29 while (l<=r){
30     int mid=(l+r>>1);
31     bool fl= ③;
32     F(i,l,mid){
33         int st= ④;
34         if (st+s[i]>tim) fl=1;
35     }
36     if (fl) r=mid-1;
37     else l=mid+1,ans=⑤;
38 }
39 cout<<ans<<"\n";
40 }
41 int main(){
42     int T=read();
43     while (T--) work();
44     return 0;
45 }

```

(1)①处应填()。

A. n-a B. n-a+1 C. a-n D. a+1

(2)②处应填()。

A. b B. b-1 C. a D. a-1

(3)③处应填()。

A. 2 B. 1 C. 0 D. 3

(4)④处应填()。

A. mid-i+1 B. l C. i+1 D. mid-i

(5)⑤处应填()。

A. mid B. l C. r D. mid+1

2. (YY 的树) YY 有一棵 n 个结点的树, 树上的每一条边可能是轻边或者重边。接下来你需要对树进行 m 次操作, 在所有操作开始前, 树上所有边都是轻边。操作有以下两种:

(1) 给定两个点 a 和 b , 首先对于 a 到 b 路径上的所有点 x (包含 a 和 b), 你要将与 x 相连的所有边变为轻边。然后再将 a 到 b 路径上包含的所有边变为重边。

(2) 给定两个点 a 和 b , 你需要计算当前 a 到 b 的路径上一共包含多少条重边。输入第一行包含两个整数 n ($1 \leq n \leq 100000$) 表示结点数量, m ($1 \leq m \leq 100000$) 表示操作数量。接下来 $n-1$ 行, 每行包含两个整数 u, v , 表示树上的一条边。

接下来 m 行, 每行包含三个整数 op_i, a_i, b_i , 描述一个操作。

其中 $op_i=1$, 表示第 1 类操作, $op_i=2$, 表示第 2 类操作,

数据保证 $a_i \neq b_i$ 。

对于每种第 2 类操作, 输出一个数表示答案。

提示: 这道题可以采用树链剖分解决。

试补全程序。

```

001 #include<bits/stdc++.h>
002 using namespace std;
003 struct edge{int nxt,to;}e[100001<<1];
004 struct tree{
005     int ans,l,r,tag,len;
006     tree(){ans=l=r=tag=len=0;}
007 }t[100001<<2];
008 int n,m,tot,h[100001],dep[100001],fa[100001],s[100001];
009 int son[100001],id[100001],cnt,top[100001];
010 void add(int x,int y){
011     e[++tot].nxt=h[x];
012     h[x]=tot;
013     e[tot].to=y;
014 }
015 int ls(int k){return k<<1;}
016 int rs(int k){return k<<1|1;}
017 tree mer(tree x,tree y){
018     tree res;
019     res.ans=x.ans+y.ans;
020     res.len=x.len+y.len;
021     if( ① )++res.ans;
022     res.l=x.l;res.r=y.r;
023     return res;
024 }
025 void pp(int k){t[k]=mer(t[ls(k)],t[rs(k)]);}
026 void pd(int k){
027     if(t[k].tag){
028         t[ls(k)].ans=t[ls(k)].len-1;
029         t[rs(k)].ans=t[rs(k)].len-1;
030         t[ls(k)].l=t[ls(k)].r=t[ls(k)].tag=t[k].tag;
031         t[rs(k)].l=t[rs(k)].r=t[rs(k)].tag=t[k].tag;
032         t[k].tag=0;
033     }
034 }
035 void bld(int k,int l,int r){
036     t[k].len= ② ;t[k].tag=t[k].ans=0;
037     if(l==r){t[k].l=t[k].r=l;return;}
038     int mid=(l+r)>>1;
039     bld(ls(k),l,mid);bld(rs(k),mid+1,r);
040     pp(k);
041 }
042 void upd(int nl,int nr,int l,int r,int k,int p){
043     if(l>=nl&&r<=nr){
044         t[k].ans= ③ ;
045         t[k].l=t[k].r=t[k].tag=p;
046         return;
    
```

```

047     }
048     pd(k);
049     int mid=(l+r)>>1;
050     if(nl<=mid)upd(nl,nr,l,mid,ls(k),p);
051     if(nr>mid)upd(nl,nr,mid+1,r,rs(k),p);
052     pp(k);
053 }
054 tree qry(int nl,int nr,int l,int r,int k){
055     if(l>=nl&&r<=nr)return t[k];
056     pd(k);
057     int mid=(l+r)>>1;
058     if(nr<=mid)return qry(nl,nr,l,mid,ls(k));
059     if(nl>mid)return qry(nl,nr,mid+1,r,rs(k));
060     return ④ ;
061 }
062 void dfs1(int k,int f,int deep){
063     dep[k]=deep;fa[k]=f;s[k]=1;son[k]=0;int maxson=-1;
064     for(int i=h[k];i;i=e[i].nxt){
065         if(e[i].to==f)continue;
066         dfs1(e[i].to,k,deep+1);
067         s[k]+=s[e[i].to];
068         if(s[e[i].to]>maxson){
069             maxson=s[e[i].to];
070             son[k]=e[i].to;
071         }
072     }
073 }
074 void dfs2(int k,int t){
075     id[k]=++cnt;top[k]=t;if(!son[k])return;
076     dfs2(son[k],t);
077     for(int i=h[k];i;i=e[i].nxt){
078         if(e[i].to==fa[k]||e[i].to==son[k])continue;
079         dfs2(e[i].to,e[i].to);
080     }
081 }
082 void up(int x,int y,int p){
083     while(top[x]^top[y]){
084         if(dep[top[x]]<dep[top[y]])x^=y^=x^=y;
085         upd(id[top[x]],id[x],1,n,1,p);
086         x=fa[top[x]];
087     }
088     if(dep[x]>dep[y])x^=y^=x^=y;
089     upd(id[x],id[y],1,n,1,p);
090 }
091 int LCA(int x,int y){
092     while(top[x]^top[y]){
093         if(dep[top[x]]<dep[top[y]])x^=y^=x^=y;

```



```

094     x=fa[top[x]];
095     }
096     if(dep[x]>dep[y])x^=y^=x^=y;
097     return x;
098 }
099 int get(int x,int lca){
100     tree res;
101     while(top[x]^top[lca]){
102         res=⑤;
103         x=fa[top[x]];
104     }
105     res=mer(qry(id[lca],id[x],1,n,1),res);
106     return res.ans;
107 }
108 int q(int x,int y){
109     int lca=LCA(x,y);
110     return get(x,lca)+get(y,lca);
111 }
112 int main(){
113     scanf("%d%d",&n,&m);
114     for(int i=1;i<n;++i){
115         int x,y;scanf("%d%d",&x,&y);
116         add(x,y);add(y,x);
117     }
118     dfs1(1,0,1);dfs2(1,1);bld(1,1,n);
119     for(int i=1;i<=m;++i){
120         int opt,x,y;scanf("%d%d%d",&opt,&x,&y);
121         if(opt==1)up(x,y,i+n);
122         if(opt==2)printf("%d\n",q(x,y));
123     }
124     return 0;
125 }

```

(1)①处应填()。

- A. x.l=y.l B. x.l=y.r C. x.r=y.l D. x.r=y.r

(2)②处应填()。

- A. r-l B. r-l+1 C. r-l+2 D. l-r

(3)③处应填()。

- A. t[k].len-1 B. t[k].len C. t[k].len+1 D. t[k].r-t[k].l+1

(4)④处应填()。

- A. (qry(nl,nr,l,mid,ls(k)).ans+qry(nl,nr,mid+1,r,rs(k)).ans)
 B. mer(qry(nl,nr,l,mid,ls(k)),qry(nl,nr,mid+1,r,rs(k)))
 C. qry(nl,nr,l,mid,ls(k)).ans
 D. qry(nl,nr,mid+1,r,rs(k)).ans

(5)⑤处应填()。

- A. mer(qry(id[x],id[top[x]],1,n,1),res)
 B. mer(qry(id[x],id[lca],1,n,1),res)
 C. mer(qry(id[lca],id[x],1,n,1),res)
 D. mer(qry(id[top[x]],id[x],1,n,1),res)